



CSM 3202: Compiler Lab

Lab Report 4: Parser(ANTLR) And Parser Bison

Name: Hasan Imam

ID: 2209020

Level: 3, Semester: 2

Bioinformatics Engineering

Bangladesh Agricultural University

Report submitted: 29th January 2026

This is a documentation to the Lab X done in the Compiler Lab Course. The report is submitted to Md. Saif Uddin, Lecturer, Department of Computer Science and Mathematics, Bangladesh Agricultural University.

1 Tasks and Implementations

1.1 Task 1: Lexer And Parser Generation using ANTLR4

1.2 Objective

To design a grammar for arithmetic expressions and use ANTLR4 to automatically generate lexer and parser components. The generated parser is used to validate expressions and produce parse trees.

1.3 Requirements

Java JDK, ANTLR4 tool, ANTLR runtime library, command line or IDE.

1.4 Installation and Setup

Download ANTLR4 tool, configure environment variables, create grammar file (.g4), and generate parser and lexer using ANTLR commands.

1.5 Implementation Repository

<https://github.com/hasan2209020/Compiler-Lab-CSM-3222>

1.6 Implementation :

listings xcolor

1.7 Grammar (ANTLR4)

Listing 1: ANTLR4 Grammar for Arithmetic Expressions

```
grammar Expr;

expr
    : expr '^' expr
    | expr ('*' | '/' ) expr
    | expr ('+' | '-' ) expr
    | '(' expr ')'
    | ID
    | NUMBER
    ;

ID : [a-zA-Z]+;
```

```
NUMBER : [0-9]+('.'[0-9]+)?;  
WS : [ \t\r\n]+ -> skip;
```

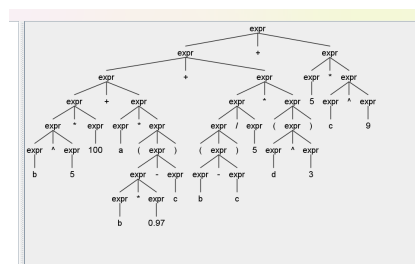
1.7.1 Input & Output

Used input.txt as input and gui based parse tree as Output .

Input:



Output:



1.8 Working Principle

ANTLR4 generates a lexer and parser from the given grammar file (.g4). The lexer converts the input string into tokens such as identifiers, numbers, and operators. The parser then checks whether these tokens follow the grammar rules.

If the input is valid, a parse tree is generated; otherwise, syntax errors are reported.

2 Task 2: Parser Design Using Flex and Bison

2.1 Objective

To design a compiler front-end component using Flex and Bison that validates while-loop syntax and arithmetic expressions based on context-free grammar rules.

2.2 Requirements

Flex, Bison, GCC or MinGW compiler, Windows/Linux environment.

2.3 Installation and Setup

Install Flex and Bison tools. Configure PATH variables. Create lexer (.l) and parser (.y) files. Compile using Flex, Bison, and GCC commands.

2.4 Implementation Repository

<https://github.com/hasan2209020/Compiler-Lab-CSM-3222>

Implementation:
listings xcolor

2.5 Implementation (Flex and Bison)

```
%{
#include <stdio.h>
#include <stdlib.h>
#include "while.tab.h"
%}

%%

"while" { return WHILE; }
"(" { return LPAREN; }
")" { return RPAREN; }
"{" { return LBRACE; }
"}" { return RBRACE; }

";" { return SEMICOLON; }
"=" { return ASSIGN; }

"==" { return EQ; }
"!=" { return NE; }
"<=" { return LE; }
">=" { return GE; }
```

```

"<" { return LT; }
">" { return GT; }

"+" { return PLUS; }
"-" { return MINUS; }
"*" { return MUL; }
"/" { return DIV; }

[0-9]+ { yylval.num = atoi(yytext); return NUMBER; }
[a-zA-Z_][a-zA-Z0-9_]* { return ID; }

[ \t\r\n]+ ;

. { printf("Unknown character: %s\n", yytext); }

%%

int yywrap() { return 1; }

%{
#include <stdio.h>
#include <stdlib.h>

void yyerror(const char *s);
int yylex();
extern FILE *yyin;
%}

%union {
    int num;
}

%token WHILE
%token LPAREN RPAREN LBRACE RBRACE
%token SEMICOLON ASSIGN
%token PLUS MINUS MUL DIV
%token EQ NE LT GT LE GE
%token <num> NUMBER
%token ID

%left PLUS MINUS
%left MUL DIV

%%

program:
    stmt_list
    ;

stmt_list:
    stmt_list stmt
    | stmt
    ;

stmt:
    while_stmt
    | assign_stmt SEMICOLON
    ;

while_stmt:
    WHILE LPAREN condition RPAREN LBRACE stmt_list RBRACE

```

```

    {
        printf("Parsing successful!\n");
    }
    ;

assign_stmt:
    ID ASSIGN expr
    ;

condition:
    expr LT expr
    | expr GT expr
    | expr LE expr
    | expr GE expr
    | expr EQ expr
    ;

expr:
    expr PLUS expr
    | expr MINUS expr
    | expr MUL expr
    | expr DIV expr
    | LPAREN expr RPAREN
    | NUMBER
    | ID
    ;

%%

void yyerror(const char *s) {
    printf("Parse Error: %s\n", s);
}

int main() {
    FILE *fp = fopen("input.txt", "r");

    if (!fp) {
        printf("Error opening input.txt\n");
        return 1;
    }

    yyin = fp;
    yyparse();

    fclose(fp);
    return 0;
}

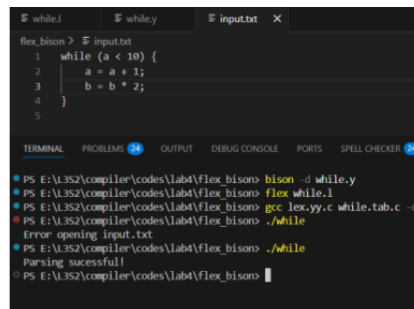
```

2.5.1 Input & Output

Input:

```
1 while (a < 10) {  
2     a = a + 1;  
3     b = b * 2;  
4 }
```

Output:



```
flex_bison > input.txt  
1 while (a < 10) {  
2     a = a + 1;  
3     b = b * 2;  
4 }  
5  
TERMINAL PROBLEMS 26 OUTPUT DEBUG CONSOLE PORTS SPELL CHECKER 34  
PS E:\352\compiler\codes\lab4\flex_bison> bison -d while.y  
PS E:\352\compiler\codes\lab4\flex_bison> flex while.l  
PS E:\352\compiler\codes\lab4\flex_bison> gcc lex.yy.c while.tab.c -o  
PS E:\352\compiler\codes\lab4\flex_bison> ./while  
Error opening input.txt  
PS E:\352\compiler\codes\lab4\flex_bison> ./while  
Parsing successful  
PS E:\352\compiler\codes\lab4\flex_bison> |
```

3 Conclusion

In this task, Flex and Bison were used to design a lexer and parser for validating while-loop syntax and arithmetic expressions. The system successfully identifies valid inputs and detects syntax errors, demonstrating the basic working of a compiler front-end.